

Docker Multi-Stage Builds, Networking, Storage & Docker Compose

1. Docker Multi-Stage Builds

The Problem with Our Existing Dockerfile

Consider this Dockerfile:

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/docker-demo-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

The important line is:

```
COPY target/docker-demo-0.0.1-SNAPSHOT.jar app.jar
```

The Dockerfile assumes that the JAR already exists inside the `target` directory.

That means someone must run:

```
mvn clean package
```

before running:

```
docker build -t docker-demo .
```

Our current workflow therefore becomes:

```
Developer Machine
  |
  v
Install JDK
  |
  v
Install Maven
  |
  v
mvn clean package
  |
  v
Generate JAR
  |
  v
docker build
  |
  v
Docker Image
```

The Docker image depends on preparation performed outside Docker.

Ideally, we want:

```
Source Code + Dockerfile
  |
  v
docker build
  |
  v
Production Image
```

The Docker build itself should compile and package the application.

2. Why Not Build Everything Inside One Docker Image?

We could create a Dockerfile like this:

```
FROM maven:3.9-eclipse-temurin-21

WORKDIR /app

COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "target/docker-demo-0.0.1-SNAPSHOT.jar"]
```

This works.

Docker can now:

1. Download Maven dependencies
2. Compile the application
3. Generate the JAR
4. Run the application

However, the final image now contains much more than the application needs at runtime.

```
Production Image
├──
├── Linux environment
├── Java JDK
├── Maven
├── Maven dependency cache
├── Source code
├── Compiled application
└── Build-related files
```

Maven and the JDK are necessary while **building** the application.

They are not necessarily required while **running** the application.

This is the problem solved by **multi-stage builds**.

3. First-Principles Idea Behind Multi-Stage Builds

Think of software creation as manufacturing.

A factory may need:

- heavy machinery
- assembly equipment
- workers
- testing equipment

But when the final product leaves the factory, we do not ship the entire factory with it.

We only ship the finished product.

The same principle applies to Docker images.

We create two separate environments.

Stage 1 — Builder

Contains everything required to:

- download dependencies
- compile source code
- package the application

Stage 2 — Runtime

Contains only what is required to execute the packaged application.

Docker allows multiple `FROM` instructions inside the same Dockerfile.

Each `FROM` starts a new build stage.

This technique is called a **multi-stage build**.

4. Creating a Multi-Stage Dockerfile

```
# ----- STAGE 1: BUILD -----  
  
FROM maven:3.9-eclipse-temurin-21 AS build  
  
WORKDIR /app  
  
COPY pom.xml .  
  
RUN mvn -B dependency:go-offline  
  
COPY src ./src  
  
RUN mvn -B -DskipTests package  
  
  
# ----- STAGE 2: RUNTIME -----  
  
FROM eclipse-temurin:21-jre AS runtime  
  
WORKDIR /app  
  
COPY --from=build /app/target/docker-demo-0.0.1-SNAPSHOT.jar app.jar  
  
EXPOSE 8080  
  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Stage 1: Build the Application

Start from a Maven + JDK Image

```
FROM maven:3.9-eclipse-temurin-21 AS build
```

This image already contains:

- Maven
- Java 21 JDK
- required build environment

`AS build` gives this stage a name.

```
build
```

Later stages can reference it.

Set the Working Directory

```
WORKDIR /app
```

All subsequent commands operate relative to `/app`.

Copy the Maven Configuration

```
COPY pom.xml .
```

Only `pom.xml` is copied initially.

This is intentionally done before copying the complete source code.

Download Dependencies

```
RUN mvn -B dependency:go-offline
```

Maven attempts to resolve the project's dependencies and relevant plugin artifacts.

- `B` enables Maven's **batch mode**, which is useful in automated environments such as Docker builds and CI pipelines.

Why Copy `pom.xml` Before the Source Code?

Docker builds images using layers.

Suppose the application source code changes, but `pom.xml` does not.

If dependency downloading happens in a layer created before the source code is copied, Docker can often reuse the cached dependency layer.

```
COPY pom.xml .  
  
RUN mvn -B dependency:go-offline  
  
COPY src ./src
```

Therefore:

```
pom.xml unchanged  
  |  
  v  
Dependency layer can be reused  
  |  
  v  
Only source code needs rebuilding
```

This can significantly improve repeated build times.

Copy the Application Source

```
COPY src ./src
```

The Java source code is now copied into the builder stage.

Package the Application

```
RUN mvn -B -DskipTests package
```

Maven compiles and packages the application.

The executable JAR is generated inside:

```
/app/target/
```

For this demo, tests are skipped.

In a real delivery pipeline, relevant automated tests should normally run before publishing the final image.

5. Stage 2: Create the Runtime Image

The second stage begins with:

```
FROM eclipse-temurin:21-jre AS runtime
```

This starts a completely new build stage.

Docker does **not** take the previous Maven image and install a JRE on top of it.

Instead, it starts from a fresh JRE image.

Conceptually:

```
Stage 1
Maven + JDK + Source Code
      |
      | Only JAR transferred
      v
Stage 2
JRE + Application JAR
```

Copy the JAR from the Builder Stage

```
COPY --from=build /app/target/docker-demo-0.0.1-SNAPSHOT.jar app.jar
```

Breakdown:

Part	Meaning
<code>COPY</code>	Copy a file into the current stage
<code>--from=build</code>	Retrieve the file from the stage named <code>build</code>
<code>/app/target/...jar</code>	Source location inside the builder stage
<code>app.jar</code>	Destination filename inside the runtime stage

Previously we used:

```
COPY target/docker-demo-0.0.1-SNAPSHOT.jar app.jar
```

That copied the JAR from our **local machine**.

Now:

```
COPY --from=build ...
```

copies the JAR from another **Docker build stage**.

Start the Application

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Only the packaged application is required in the final runtime image.

6. Add a `.dockerignore`

Create:

```
.dockerignore
```

Example:

```
target/  
.git/  
.idea/  
*.iml  
*.log  
.env
```

Docker sends a **build context** when building an image.

Files that are not needed during the build should be excluded.

Our local `target` directory is especially unnecessary because the builder stage generates its own JAR.

Project structure:

```
docker-demo/  
|  
├─ Dockerfile  
├─ .dockerignore  
├─ pom.xml  
└─ src/
```

7. Testing the Multi-Stage Build

Delete the locally generated Maven output:

```
rm -rf target
```

Now build the image:

```
docker build -t docker-demo:v3 .
```

Even though no local JAR exists, Docker can build the application.

The build process becomes:

1. Pull Maven + JDK image
2. Copy pom.xml
3. Download Maven dependencies
4. Copy source code
5. Compile application
6. Generate executable JAR
7. Start fresh JRE stage
8. Copy generated JAR
9. Produce final runtime image

Run the container:

```
docker run -d \
  --name demo-v3 \
  -p 8082:8080 \
  docker-demo:v3
```

Verify:

```
curl http://localhost:8082/hello
```

Expected response:

```
Hello from Docker!
```

8. Verify That Maven Is Not in the Final Image

Run:

```
docker run --rm \
  --entrypoint sh \
  docker-demo:v3 \
  -c "command -v mvn || echo 'Maven not found'"
```

Expected output:

```
Maven not found
```

This demonstrates the main benefit of a multi-stage build:

Build tools are available during the build but do not need to exist in the final runtime image.

9. Docker Networking

Why Do Containers Need Networking?

Consider a Spring Boot application running on port `8080`.

```
Spring Boot Application
|
Port 8080
```

Now the application needs MySQL:

```
Spring Boot Application
|
|
MySQL Database
Port 3306
```

When both applications run directly on the same machine, Spring Boot may connect using:

```
spring.datasource.url=jdbc:mysql://localhost:3306/shopdb
```

This works because both processes share the same host networking environment.

Now place them in separate containers:

Container A ----- Spring Boot Port 8080	Container B ----- MySQL Port 3306
--	--

If Spring Boot still uses:

```
spring.datasource.url=jdbc:mysql://localhost:3306/shopdb
```

the connection fails.

Why?

Because every container normally gets its own isolated networking environment.

```

Ubuntu Host
|
├─ Container A
|   └─ localhost → Container A
└─ Container B
    └─ localhost → Container B

```

Inside the Spring Boot container, `localhost` refers to the Spring Boot container itself.

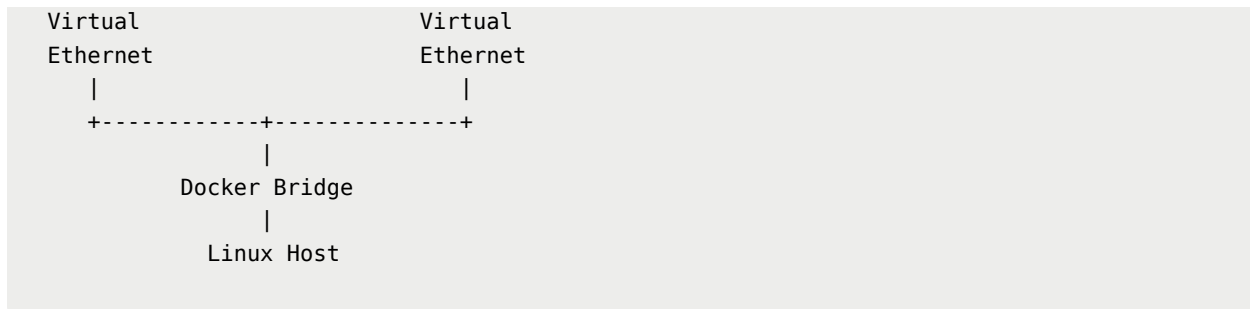
It does not refer to the MySQL container.

10. How Docker Networking Works

On Linux, Docker can connect containers using virtual network interfaces and virtual bridges.

Conceptually:

Container A 172.17.0.2 eth0 	Container B 172.17.0.3 eth0
--	--



A virtual Ethernet pair, commonly called a **veth pair**, connects a container's network namespace to networking infrastructure on the host.

Docker handles most of this configuration automatically.

11. Docker Networking Modes

Three important networking modes are:

Bridge
Host
None

Bridge Networking

Containers communicate through a virtual network.

This is the most common model for standalone multi-container applications.

Host Networking

The container uses the host's networking environment directly.

None Networking

The container has no normal external network connectivity.

12. Bridge Networking

Bridge networking provides containers with separate networking environments while connecting them through a virtual bridge.

When a container is started without explicitly selecting a network, Docker normally attaches it to the default `bridge` network.

View available networks:

```
docker network ls
```

Typical output:

NETWORK ID	NAME	DRIVER	SCOPE
abc123	bridge	bridge	local
def456	host	host	local
ghi789	none	null	local

Containers normally have:

- their own network namespace
- their own internal IP address
- their own ports

Those container ports can optionally be published to ports on the host.

13. Default Bridge vs User-Defined Bridge

Suppose Spring Boot needs to communicate with MySQL.

One option would be to inspect the MySQL container IP:

```
spring.datasource.url=jdbc:mysql://172.17.0.3:3306/shopdb
```

This is a poor application-level solution.

Container IP addresses are not reliable identifiers.

A recreated container may receive another IP address.

Instead, we want:

```
spring.datasource.url=jdbc:mysql://shop-db:3306/shopdb
```

This is where **user-defined bridge networks** are useful.

Docker provides automatic container-name DNS resolution on user-defined bridge networks.

Therefore containers can communicate using meaningful names instead of hard-coded IP addresses.

14. Creating a User-Defined Network

Create a network:

```
docker network create shop-network
```

Start MySQL:

```
docker run -d \  
  --name shop-db \  
  --network shop-network \  
  -e MYSQL_ROOT_PASSWORD=demo-root-pass \  
  -e MYSQL_DATABASE=shopdb \  
  mysql:8.4
```

Spring Boot can now use:

```
spring.datasource.url=jdbc:mysql://shop-db:3306/shopdb
```

Both containers must belong to the same user-defined network.

Notice that MySQL's port does not have to be published using:

```
-p 3306:3306
```

if only other containers on the same network need to access it.

Container-to-container communication can happen directly through the internal port.

15. Host Networking

With bridge networking, containers have their own isolated networking environments.

Host networking behaves differently.

On native Linux Docker:

```
docker run -d \
  --name host-web \
  --network host \
  nginx:alpine
```

The container shares the host's network namespace.

Conceptually:

```
Linux Host Network
  |
  +---- Host Processes
  |
  +---- Container Process
```

With host networking, port publishing is generally unnecessary because the process uses the host networking environment directly.

Host networking may reduce some networking overhead, but it also reduces network isolation.

It should therefore not be treated as the default choice for every container.

16. None Networking

Sometimes a container should operate without normal external networking.

Examples:

- local file processing
- isolated computation
- offline tasks
- security-sensitive processing that does not require networking

Run:

```
docker run -it \  
  --network none \  
  alpine sh
```

Inside the container:

```
ip addr
```

You should see the loopback interface but no normal external network interface.

Trying:

```
wget https://example.com
```

should fail because external network connectivity has not been configured.

17. Docker Storage

Networking solves container communication.

Storage solves another problem:

| What happens to application data when a container is removed?

Consider a MySQL container containing:

```
Customers: 500  
Products: 100
```

If the database container is removed, important data should not disappear with it. To understand why this can happen, we need to understand container storage.

18. Container Writable Layer

Docker images contain read-only layers.

When Docker creates a container from an image, it adds a writable container layer.

```
graph TD; A[Docker Image  
Read-only layers] --> B[Container  
Writable layer]
```

The diagram illustrates the relationship between a Docker image and a container. It shows a 'Docker Image' composed of 'Read-only layers'. An arrow points down from these layers to a 'Container', which has its own 'Writable layer'.

Files created or modified in the ordinary container filesystem are represented through this writable layer.

The writable layer belongs to that specific container.

Therefore:

- Stopping a container does **not** delete its writable layer.
- Starting the same stopped container retains its files.
- Removing the container removes its writable layer.

Important application data should therefore not depend entirely on this temporary container layer.

19. Docker Volumes

A Docker volume is storage managed separately from the lifecycle of an individual container.

Conceptually:

```
Container
  |
  | mounted
  v
Docker Volume
  |
  v
Host Storage
```

The container can be:

- stopped
- removed
- recreated

while the volume continues to exist.

A volume is essentially passive storage managed by Docker and mounted into containers.

20. Named Volumes

A named volume has an explicit name.

Create one:

```
docker volume create my-data
```

List volumes:

```
docker volume ls
```

Example:

```
DRIVER    VOLUME NAME
```

```
local    my-data
```

Attach it to a container:

```
docker run -it \  
  -v my-data:/app/data \  
  alpine sh
```

Syntax:

```
-v VOLUME_NAME:CONTAINER_PATH
```

Here:

```
my-data    → Docker volume  
/app/data → Mount location inside the container
```

Anything written to:

```
/app/data
```

is stored in the mounted volume rather than relying only on the container's writable layer.

21. Docker Compose

The Problem Compose Solves

A real application may require multiple containers.

For example:

```
Spring Boot Application  
|  
+----- MySQL
```

```
|  
+---- Redis
```

Using plain Docker might require creating a network first:

```
docker network create shop-network
```

Starting MySQL:

```
docker run -d \  
  --name shop-db \  
  --network shop-network \  
  -e MYSQL_ROOT_PASSWORD=root \  
  -e MYSQL_DATABASE=shopdb \  
  -v mysql-data:/var/lib/mysql \  
  mysql:8.4
```

Starting Redis:

```
docker run -d \  
  --name shop-redis \  
  --network shop-network \  
  redis:alpine
```

Starting Spring Boot:

```
docker run -d \  
  --name shop-app \  
  --network shop-network \  
  -p 8080:8080 \  
  -e SPRING_DATASOURCE_URL=jdbc:mysql://shop-db:3306/shopdb \  
  -e SPRING_DATASOURCE_USERNAME=root \  
  -e SPRING_DATASOURCE_PASSWORD=root \  
  shop-app
```

Technically this works.

But anyone running the application now needs to know:

- which containers are required

- which images are required
- which ports need to be published
- which environment variables are required
- which networks must exist
- which volumes are required
- how containers depend on each other

The infrastructure becomes a long collection of commands stored in a README or someone's memory.

Docker Compose solves this problem.

22. What Is Docker Compose?

Docker Compose allows us to describe a multi-container application using a configuration file.

The file is usually:

```
compose.yaml
```

Inside it, we can define:

```
Application
├── Services
│   ├── app
│   ├── database
│   └── redis
├── Networks
└── Volumes
```

Instead of executing many separate commands:

```
docker network create ...
docker volume create ...
docker run ...
docker run ...
docker run ...
```

we describe the application once and run:

```
docker compose up
```

Conceptually:

```
Developer
  |
  v
compose.yaml
  |
  v
Docker Compose
  |
  v
Docker Engine
  |
  +---- Containers
  +---- Networks
  +---- Volumes
  +---- Images
```

Compose acts as an orchestration and configuration layer for a group of Docker resources.

23. Why Docker Compose Is Declarative

Plain `docker run` commands are largely imperative.

We tell Docker:

```
Create this network.
Create this volume.
```

```
Start this container.  
Start another container.
```

Compose is more declarative.

Instead of focusing on the sequence of commands, we describe the desired structure.

```
services:  
  
  app:  
    ...  
  
  db:  
    ...  
  
volumes:  
  
  mysql-data:
```

We describe:

| This is what my application should look like.

Compose then manages the required Docker resources.

24. Our First Compose File

Create:

```
compose.yaml
```

Add:

```
services:  
  
  web:  
    image: nginx:alpine
```

```
ports:
  - "8080:80"
```

Run:

```
docker compose up
```

Compose understands that:

- a service named `web` is required
- it uses the `nginx:alpine` image
- container port `80` should be published through host port `8080`

Open:

```
http://localhost:8080
```

To run in the background:

```
docker compose up -d
```

25. Understanding Services

The most important section of a Compose file is:

```
services:
```

Example:

```
services:
  web:
    image: nginx:alpine
```

`web` is a **service name**.

A service represents a logical component of the application.

For example:

```
services:
  frontend:
  backend:
  database:
  redis:
```

Conceptually:

```
Compose Application
├─ frontend
├─ backend
├─ database
└─ redis
```

A service normally results in one or more containers created using the same configuration.

26. Basic Compose File Structure

A typical Compose file can contain:

```
services:
  ...

networks:
  ...

volumes:
  ...
```

The most important top-level sections for common applications are:

- `services`
- `networks`
- `volumes`

You may also encounter:

- `configs`
- `secrets`

27. YAML Lists

Example:

```
ports:
  - "8080:80"
  - "8081:81"
```

The `-` represents an item inside a YAML list.

28. `image` vs `build`

This distinction is important.

If we want MySQL, we do not need to build MySQL ourselves.

We can use:

```
services:
  db:
    image: mysql:8.4
```

`image` means:

| Use an existing Docker image.

Our Spring Boot application is different because we own its source code and Dockerfile.

Compose can build it using:

```
services:
  app:
    build: .
```

`build: .` means:

Use the current directory as the build context and build the application using its Dockerfile.

Summary

Property	Meaning
<code>image</code>	Use an existing Docker image
<code>build</code>	Build an image from a Dockerfile

29. Spring Boot Project with Compose

A project may look like:

```
shop-app/
|
├─ Dockerfile
├─ compose.yaml
├─ pom.xml
└─ src/
```

Because our Dockerfile uses a multi-stage build, a pre-built local `target` directory is not required for the Docker image build.

30. Compose Networking

One of the most useful features of Compose is automatic networking.
With plain Docker, we previously created a network manually:

```
docker network create shop-network
```

and attached containers:

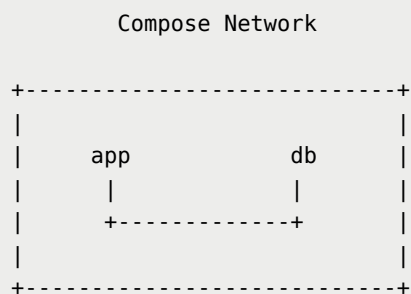
```
docker run --network shop-network ...
```

Compose normally creates a default network for the project automatically.
Services in the same Compose application are connected to it.

For example:

```
services:
  app:
  db:
```

Conceptually:



Services can discover each other using their **service names**.

Therefore the application can connect to MySQL using:

```
db
```

as the hostname.

Example:

```
jdbc:mysql://db:3306/shopdb
```

No hard-coded container IP address is required.

31. Environment Variables in Compose

With plain Docker we might write:

```
-e MYSQL_ROOT_PASSWORD=root  
-e MYSQL_DATABASE=shopdb
```

Compose equivalent:

```
services:  
  
  db:  
    image: mysql:8.4  
  
    environment:  
      MYSQL_ROOT_PASSWORD: root  
      MYSQL_DATABASE: shopdb
```

For Spring Boot:

```
app:  
  
  build: .  
  
  environment:  
    SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/shopdb  
    SPRING_DATASOURCE_USERNAME: root  
    SPRING_DATASOURCE_PASSWORD: root
```

Compose passes these environment variables to the containers when they are created.

32. Volumes in Docker Compose

Database state should not depend on the container's writable layer.

For MySQL, create a named volume through Compose:

```
services:
  db:
    image: mysql:8.4
    volumes:
      - mysql-data:/var/lib/mysql

volumes:
  mysql-data:
```

Two different declarations are involved.

Service-Level Volume Mount

```
volumes:
  - mysql-data:/var/lib/mysql
```

This means:

Mount the `mysql-data` volume inside this container at `/var/lib/mysql`.

MySQL stores its database files at:

```
/var/lib/mysql
```

Top-Level Volume Declaration

```
volumes:  
  
  mysql-data:
```

This declares a named volume for the Compose application.

The volume's lifecycle can remain independent of the database container.

33. Container Started vs Application Ready

A common problem appears when starting multiple services.

Suppose Compose starts MySQL and then Spring Boot immediately tries to connect.

Spring Boot may produce:

```
Connection refused  
Could not connect to MySQL
```

Why?

Because:

```
Container Started  
  ≠  
Application Ready
```

Starting the MySQL container only means the container process has started.

MySQL itself may still need time to:

- initialize files
- create system tables
- create configured databases
- configure users

- start listening for connections

This distinction between **process started** and **service ready** is important throughout DevOps.

34. `depends_on`

Compose allows dependencies between services.

Example:

```
app:
  depends_on:
    - db
```

This expresses:

```
app depends on db
```

With the short syntax, Compose starts the dependency before the dependent service.

Conceptually:

```
Start db
  |
  v
Start app
```

However, this does **not** necessarily mean that MySQL is ready to accept connections.

`depends_on` by itself does not completely solve the readiness problem.

35. Health Checks

We can define a health check for MySQL.

```
db:

  image: mysql:8.4

  healthcheck:

    test:
      [
        "CMD-SHELL",
        "mysqladmin ping -h localhost -u root -p$$MYSQL_ROOT_PASSWORD --silent"
      ]

    interval: 5s
    timeout: 5s
    retries: 10
    start_period: 20s
```

The health check gives Docker a way to determine whether MySQL is actually responding.

test

```
test:
```

Defines the command Docker executes to determine the health of the container.

Here:

```
mysqladmin ping
```

checks whether MySQL responds.

interval

```
interval: 5s
```

Run the health check periodically.

timeout

```
timeout: 5s
```

If a health-check attempt takes longer than this, the attempt is considered failed.

retries

```
retries: 10
```

Repeated failed attempts can cause the container to be marked as unhealthy.

start_period

```
start_period: 20s
```

Provides an initialization grace period before failures begin counting toward the retry limit.

36. Wait for MySQL to Become Healthy

We can combine `depends_on` with the health status:

```
app:
  depends_on:
    db:
      condition: service_healthy
```

Now the startup lifecycle becomes:

```
Create DB Container
  |
  v
Start DB
  |
  v
Run Health Checks
  |
  v
MySQL Healthy
  |
  v
Start Application
```

This is much safer than relying only on container startup order.

37. Complete Spring Boot + MySQL Compose Example

A simplified Compose configuration may look like:

```
services:

  db:

    image: mysql:8.4

    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: shopdb

    volumes:
      - mysql-data:/var/lib/mysql

    healthcheck:
      test:
        [
          "CMD-SHELL",
          "mysqladmin ping -h localhost -u root -p$MYSQL_ROOT_PASSWORD --silent"
        ]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 20s
```

```

app:

  build: .

  ports:
    - "8080:8080"

  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/shopdb
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: root

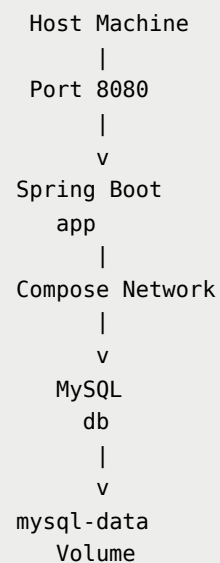
  depends_on:
    db:
      condition: service_healthy

  volumes:

    mysql-data:

```

The architecture becomes:



Notice that the application connects to:

```
db:3306
```

rather than:

```
localhost:3306
```

because `db` is the MySQL service name inside the Compose network.

38. Important Docker Compose Commands

A small set of commands covers most day-to-day Compose usage.

Start All Services

```
docker compose up
```

Starts the application stack and shows logs in the current terminal.

Start in Detached Mode

```
docker compose up -d
```

Starts the application in the background.

Build and Start

```
docker compose up --build
```

Useful when:

- application source changes

- the Dockerfile changes
- a new image needs to be built

View Compose Containers

```
docker compose ps
```

Example:

NAME	SERVICE	STATUS
shop-app-1	app	running
shop-db-1	db	healthy

View Logs

For all services:

```
docker compose logs
```

Follow logs continuously:

```
docker compose logs -f
```

39. docker compose stop vs docker compose down

These commands have different purposes.

Stop

```
docker compose stop
```

Stops the containers but keeps them.

Conceptually:

```
Containers → stopped but still exist  
Resources → retained
```

They can be started again using:

```
docker compose start
```

Down

```
docker compose down
```

This removes more of the Compose application's runtime resources.

Conceptually:

```
Service Containers → removed  
Project Network → removed
```

Named volumes are **not removed by default**.

This behaviour is important because database data should normally survive application container recreation.

40. Mental Model of Docker Compose

Without Compose:

```
Developer
|
+---- Create network
|
+---- Create volume
|
+---- Start MySQL
|
+---- Configure MySQL
|
+---- Start Redis
|
+---- Start Spring Boot
|
+---- Configure ports
|
+---- Configure environment variables
```

With Compose:

```
Developer
|
v
compose.yaml
|
v
docker compose up
|
v
+-----+
| Docker Compose |
|               |
| Creates containers |
| Creates network   |
| Creates/uses volumes |
| Applies configuration |
| Handles dependencies |
+-----+
```

The biggest advantage is not simply shorter commands.

The real advantage is that the **entire local application stack becomes reproducible configuration.**

Anyone with:

```
Source Code
Dockerfile
compose.yaml
```

can reproduce the same containerized environment.

41. Final Picture

These concepts solve different parts of the containerization problem.

```
Multi-Stage Builds
  |
  v
Build application inside Docker
while keeping runtime image clean
```

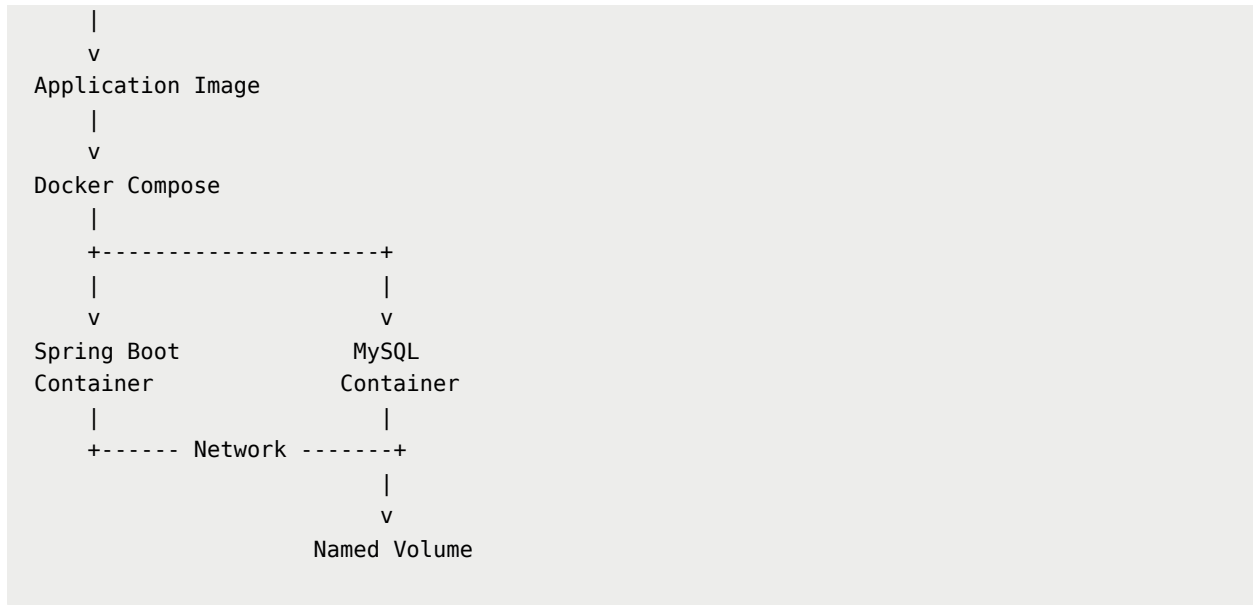
```
Docker Networking
  |
  v
Allow containers to communicate
```

```
Docker Volumes
  |
  v
Keep important data outside
the container lifecycle
```

```
Docker Compose
  |
  v
Define and run the complete
multi-container application
from one configuration file
```

Together they give us a much more realistic application architecture:

```
Source Code
  |
  v
Multi-Stage Dockerfile
```



Key Takeaways

- A normal Dockerfile may depend on a JAR being created outside Docker.
- Multi-stage builds separate the **build environment** from the **runtime environment**.
- The builder stage can contain Maven, JDK, source code, and build tools.
- The runtime stage only needs the final application and runtime dependencies.
- Containers normally have isolated networking environments.
- `localhost` inside one container refers to that container itself.
- User-defined bridge networks allow containers to communicate using names.
- Important persistent data should not live only in a container's writable layer.
- Docker volumes provide storage with a lifecycle independent of individual containers.
- Docker Compose describes multi-container applications using `compose.yaml`.
- Compose automatically provides networking for services in the same project.
- Service names such as `db` can be used as hostnames.

- `depends_on` controls dependency ordering, but startup does not necessarily mean readiness.
- Health checks allow Compose to wait until services such as MySQL are actually healthy.
- `docker compose stop` keeps containers, while `docker compose down` removes service containers and project networks.
- Named volumes survive `docker compose down` by default, helping preserve persistent data.